

Evaluating the Effectiveness of LLMs in Introductory Computer Science Education: A Semester-Long Field Study

Wenhan Lyu
William & Mary
Williamsburg, VA, USA
wlyu@wm.edu

Yimeng Wang
William & Mary
Williamsburg, VA, USA
ywang139@wm.edu

Tingting (Rachel) Chung
William & Mary
Williamsburg, VA, USA
rachel.chung@mason.wm.edu

Yifan Sun
William & Mary
Williamsburg, VA, USA
ysun25@wm.edu

Yixuan Zhang
William & Mary
Williamsburg, VA, USA
yzhang104@wm.edu

ABSTRACT

The integration of AI assistants, especially through the development of Large Language Models (LLMs), into computer science education has sparked significant debate, highlighting both their potential to augment student learning and the risks associated with their misuse. An emerging body of work has looked into using LLMs in education, primarily focusing on evaluating the performance of existing models or conducting short-term human subject studies. However, very little work has examined the impacts of LLM-powered assistants on students in entry-level programming courses, particularly in real-world contexts and over extended periods. To address this research gap, we conducted a semester-long, between-subjects study with 50 students using CodeTutor, an LLM-powered assistant developed by our research team. Our study results show that students who used CodeTutor (the “CodeTutor group” as the experimental group) achieved statistically significant improvements in their final scores compared to peers who did not use the tool (the “control group”). Within the CodeTutor group, those without prior experience with LLM-powered tools demonstrated significantly greater performance gain than their counterparts. We also found that students expressed positive feedback regarding CodeTutor’s capability to comprehend their queries and assist in learning programming language syntax. However, they had concerns about CodeTutor’s limited role in developing critical thinking skills. Over the course of the semester, students’ agreement with CodeTutor’s suggestions decreased, with a growing preference for support from traditional human teaching assistants. Our findings also show that students turned to CodeTutor for different tasks, including programming task completion, syntax comprehension, and debugging, particularly seeking help for programming assignments. Our analysis further reveals that the quality of user prompts was significantly correlated with CodeTutor’s response effectiveness. Building upon these results, we discuss the implications of our findings for the need to integrate Generative AI literacy into curricula to foster critical thinking skills, and turn to examining the temporal dynamics

of user engagement with LLM-powered tools. We further discuss the discrepancy between the anticipated functions of tools and students’ actual capabilities, which sheds light on the need for tailored strategies to improve educational outcomes.

CCS CONCEPTS

• **Human-centered computing** → **Human computer interaction (HCI)**.

KEYWORDS

Field study, Large Language Models, Tutoring

ACM Reference Format:

Wenhan Lyu, Yimeng Wang, Tingting (Rachel) Chung, Yifan Sun, and Yixuan Zhang. 2024. Evaluating the Effectiveness of LLMs in Introductory Computer Science Education: A Semester-Long Field Study. In *Proceedings of the Eleventh ACM Conference on Learning @ Scale (L@S '24)*, July 18–20, 2024, Atlanta, GA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3657604.3662036>

1 INTRODUCTION

Recent advancements in Generative AI and Large Language Models (LLMs), exemplified by GitHub Copilot [15] and ChatGPT [32], have demonstrated their capacity to tackle complex problems with human-like proficiency. These innovations raise significant concerns within the educational domain, particularly as students might misuse these tools, thereby compromising the quality of education and breaching academic integrity norms [36]. Specifically, entry-level computer science education is directly affected by the progress in LLMs [59]. LLMs’ capability in handling programming tasks means they can complete many assignments typically given in introductory courses, thus becoming highly appealing to students looking for easy solutions.

Despite these challenges, LLM-powered tools offer great opportunities to enrich computer science education [23]. When used ethically and appropriately, they can serve as powerful educational resources. For instance, LLMs can provide students instant feedback on their coding assignments or generate diverse examples of code that help demonstrate programming concepts [35]. Moreover, as Generative AIs are becoming popular in production environments, familiarizing students with these technologies is increasingly becoming a crucial aspect of computer science education.



This work is licensed under a Creative Commons Attribution International 4.0 License.

L@S '24, July 18–20, 2024, Atlanta, GA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0633-2/24/07.
<https://doi.org/10.1145/3657604.3662036>

The unique challenges posed by LLMs stem from the difficulty in detecting the use of AI tools [54, 58]. Traditional approaches, such as plagiarism detection software, fall short in determining the originality of student submissions [28]. Given the challenges in identifying LLMs usage and recognizing the potential advantages of these technologies, we consider integrating LLMs into computer science education inevitable. As students have already started using such tools, the impact of LLMs on computer science education remains unknown. Indeed, a growing body of research has begun to explore the application of LLMs within educational settings, primarily focusing on assessing the capabilities of current models with existing datasets or previous assignments from students [18, 27]. However, there is still a research gap in understanding how students interact with LLM-powered tools in introductory programming classes, particularly regarding their engagement in genuine learning settings over extended periods. Furthermore, while previous studies have shown individual differences in intelligent tutoring systems [22], research into how these differences apply to LLM tools is lacking. Investigating these variations is important for tailoring educational strategies to diverse student needs. In short, understanding these nuanced attitudes of and interactions with LLM-powered tools in CS education over extended periods is crucial for identifying the evolving challenges and opportunities LLMs introduce.

To address the research gap, we asked the following research questions (RQs) in this work:

RQ1. Does the integration of LLM-powered tools in introductory programming courses enhance or impair students' learning outcomes, compared to traditional teaching methods? How are individual differences associated with students' learning outcomes using LLM-powered tools?

RQ2. What are students' attitudes towards LLM-powered tools, how do they change over time, and which factors might influence these attitudes?

RQ3. How do students engage with LLM-powered tools, and how do they respond to their programming needs?

We believe that addressing the following research questions (RQs) is critical for enabling researchers to make informed decisions about incorporating LLMs into their courses and guiding students on the optimal and responsible use of LLM-powered tools. To answer the questions, we conducted a longitudinal, between-subject field study with 50 students over the course of the fall semester from September to December 2023 with a web-based tool we developed called CodeTutor.

The **contributions** of this work are: **1)** We conducted a semester-long longitudinal field study to assess the effectiveness of an LLM-powered tool (CodeTutor) on students' learning outcomes in an introductory programming course. By comparing the performance of students who used CodeTutor against those who did not, our study contributes to new empirical evidence regarding the role of LLM-powered tools in the programming learning experience; **2)** We characterized patterns of student engagement with CodeTutor and analyzed the ways in which it can meet students' learning needs. Through the analysis of conversational interactions and feedback loops between students and the tool, we contributed new knowledge regarding how CodeTutor facilitates or impedes learning; and **3)** We offered insights and outlined design implications for future research.

2 RELATED WORK

2.1 Intelligent Tutoring Systems

Using computerized tools for assisting educational purposes is not a new idea. As early as the 1950s, the first concept of using computers to assist learning has already emerged [29]. From where the factor of intelligence had been considered and it had started evolving into *Intelligent Tutoring Systems (ITS)* [46]. ITS leverages artificial intelligence to provide personalized learning experiences in computer science education, adapting instruction and feedback to individual student needs [3, 14]. These systems have enhanced student engagement, comprehension, and problem-solving skills by offering tailored support and immediate feedback, similar to one-on-one tutoring [10, 52]. Research has demonstrated that ITS can significantly improve understanding of complex concepts in programming courses compared to traditional teaching methods, leading to higher student satisfaction due to the personalized learning environment [9, 42]. The Internet also empowered ITS to offer more interactivity and adaptivity [5–7], leveraging the path of later boost with natural language processing techniques [13, 19].

However, prior work has shown that as the granularity of tutoring decreases, its effectiveness increases [52]. Significant limitations for ITS include the complexity and cost of building them, the incapability to answer questions and tasks out of their programmed domains, and the difficulty to develop with the purpose of productively used by individuals without expertise [16]. Even though the Generalized Intelligent Framework for Tutoring (GIFT) framework [47] was proposed and evolved for developing ITS for use at scale, those limitations mostly remain unresolved.

2.2 Large Language Models in CS Education

The release of ChatGPT and other Generative AI applications brought LLMs into the public view and attracted enormous attention [1, 48]. LLMs offer researchers and users the flexibility to employ a single tool across various tasks [53], such as medical research [8, 49], finance [55], and education [21]. Adopting LLM-powered tools in educational settings is facilitated by their broad accessibility and cost-free nature [56]. Recent studies have looked into the potential of AI assistants to enhance student learning by helping with students' problem-solving [2, 25, 37] and generating computer science content [11, 43]. Current research on the use of LLMs in education has primarily looked into their performance and capabilities [40] compared to humans, such as generating code for programming tasks [24, 39], answering general inquiries [38, 44], addressing textbook questions [20] and exam questions [12].

Despite the growing interest in examining the capabilities of LLMs in education, very few empirical studies have examined the emerging concerns regarding their impact. Therefore, there is an urgent need for research into the long-term effects of LLMs in CS education and the development of strategies to counteract potential negative consequences. One exceptional work was conducted by Liffiton et al. [26], who developed a tool called CodeHelp for assisting students with their debug needs in an undergraduate course over 12 weeks. Their follow-up study [45] categorized history messages in their tool, and found a positive relationship between tool usage and course performance. However, their study specifically

focused on debugging issues and did not compare the outcomes with those achieved through traditional TA methods.

Furthermore, prior research has demonstrated that individual differences, such as gender, race, and prior experiences with technologies, significantly influence the effectiveness of intelligent tutoring systems [22]. However, work that examines how individual differences affect interactions with and perceptions of LLM-powered tools in educational settings is sparse, even though understanding the role of demographic and individual variability is crucial [57]. This is particularly important for developing inclusive and effective educational tools that suit the diverse needs of students.

Our work seeks to address these research gaps by conducting a field study that evaluates the use of LLM-powered tools for an extended period of time. Particularly, our study not only aims to evaluate the practicality of LLMs in programming learning educational contexts, but also intends to contribute to a more nuanced understanding of their long-term implications for learning and teaching methodologies.

3 METHOD

In this section, we describe the design of CodeTutor (subsection 3.1), an overview of our participants (subsection 3.2), our study procedure and data collection (subsection 3.3), and our quantitative and qualitative data analysis (subsection 3.4). The source code of CodeTutor, pre-test questions, and data analysis code is available on osf.io/e3zgh.

3.1 Design of CodeTutor

We developed CodeTutor, a browser-based web application using TypeScript and front-end frameworks (e.g., SolidJS, Astro, and libraries such as Zag), for a responsive and interactive user interface. CodeTutor integrates OpenAPI API, which enables the GPT-3.5 model offered by OpenAI. The main interface is shown in Figure 1. **Login.** Students log in to CodeTutor using their email addresses, with a randomly generated unique identifier (UID) that tracks their activities anonymously.

User Interface. The CodeTutor interface features a navigation sidebar and a central chat area. The sidebar enables easy navigation, with a button for starting new conversations and a chronological listing of existing ones for quick access.

User Feedback Structure. Feedback is important in CodeTutor in order to understand user engagement and students' attitudes towards it. CodeTutor provides two feedback mechanisms: 1) conversation-level and 2) message-level feedback.

Data Storage. CodeTutor stores data locally on the user's browser with IndexedDB and can only *upload* essential information with our secure server for research purposes, where a unique ID for anonymous tracking identifies each conversation. To protect privacy, CodeTutor cannot read stored data from our server.

API Usage. OpenAI only offered limited configuration ability for their API at the time we started our experiment. So we carefully crafted the *system role* text in our implementation to specify the model to answer questions as a teaching assistant in an entry-level Python class, making answers from OpenAI API consistent even if the length of a conversation exceeds its token limit.

3.2 Participants

Upon approval from our institution's Institutional Review Board (IRB), we conducted a field study evaluation study with 50 participants. The field study took place in the Computer Science Department of a 4-year university in the United States. Our criteria for participation include: Participants need to be 18 years or older, be able to speak and write in English, and register as entry-level undergraduate computer science students at our institution. Table 1 presents an overview of our participants' demographic information.

Table 1: Overview of Participant Characteristics

Characteristics	Options	Number of participants
Gender	Woman	22
	Man	25
	Non-binary	1
	Prefer not to say	2
Major	Computer Science	18
	Data Science	9
	Biology	5
	Mathematics	4
	Economics	3
	Others	10
	Not reported	1
Year of Study	Freshman	37
	Sophomore	5
	Junior	6
	Senior	1
	Not reported	1
Race	African American or Black	1
	Asian	17
	Multiracial	3
	White	26
	Not reported	3
Ethnicity	Latino/Hispanic	3
Prior Experience with LLM tools	Only ChatGPT	28
	ChatGPT and other tools	11
	Never used	11

3.3 Study Procedure & Data Collection

Our field study lasted from September 27 (after the course add-drop period) to December 11, 2023 (the final exam due). Below, we describe each component of our study.

3.3.1 Pre-test. Participants were initially requested to provide their consent to participate, with being informed about the study's objectives, procedures, and their rights as participants, including the right to withdraw at any time without penalty. Following the consent process, the pre-test assessment was administered to evaluate students' existing knowledge of Python programming, providing a baseline for subsequent analysis.

This pre-test included three sections with Python questions, with a total of 22 questions that varied in difficulty for an evaluation of participant skills. The first section featured eight questions (Questions 1-8, for example, "What is the output of the following code: `print(3+4)`?"), the second section included seven questions of medium difficulty (Questions 9-15, for example, "If I wanted a function to return the product of two numbers *a* and *b*, what should the return statement look like?"), and the third section presented

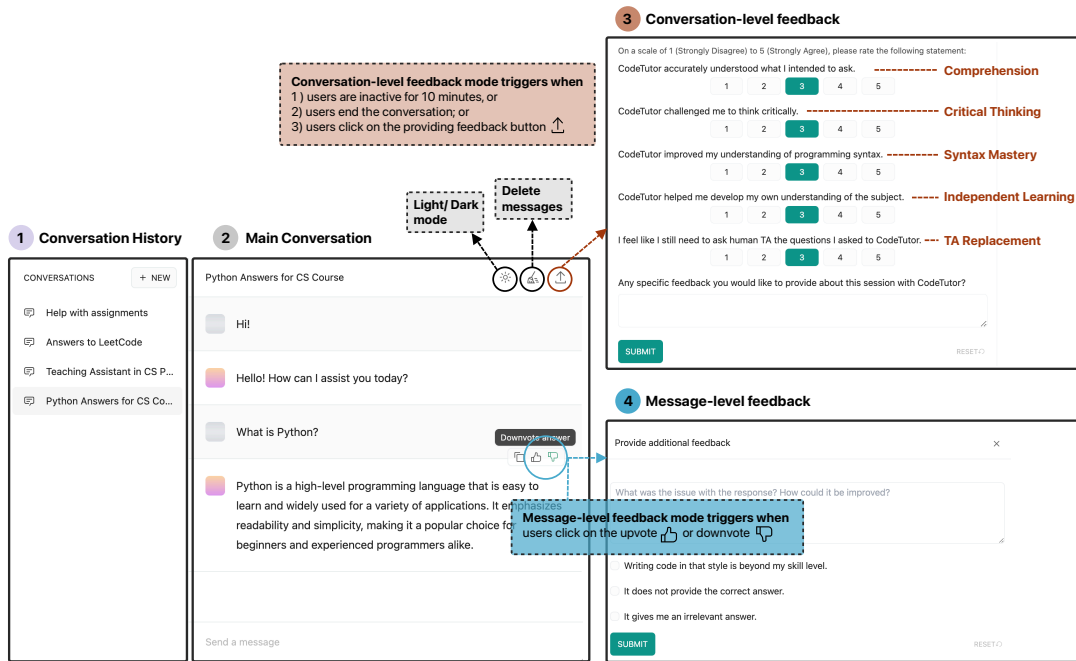


Figure 1: CodeTutor is a web application that leverages OpenAI API, featuring four main components: ① **Conversation History** that lists different conversation threads, ② **Main Conversation** that shows an ongoing dialogue with CodeTutor, ③ **Conversation-level Feedback** module that allows users to elaborate on their attitudes towards CodeTutor by proving ratings on 1) comprehension, 2) critical thinking, 3) syntax mastery, 4) independent learning, and 5) TA replacement likelihood, and to provide specific comments, and ④ **Message-level Feedback** that offers options for users to give detailed feedback on individual messages or responses from CodeTutor.

seven challenging questions (Questions 16-22, for example, “What will be the output of the following code? [Multiple lines of code]”). The total score of the three sections was 100 points. Pre-test submissions were graded by our researchers with Computer Science backgrounds, using predetermined scoring criteria.

This pre-test also asked about participants’ prior experience with LLMs, specifically asking, “Which of the following Large Language Model AI tools have you used before? Please select all that apply.” Participants were also asked to provide demographic information, including their major (or intended major), gender, and race/ethnicity. Participants were assured that all demographic information would remain anonymous and be used solely for research purposes.

3.3.2 Control vs. Experimental Group. Participants were divided into two groups: the control group, which used traditional learning methods and had access to human teaching assistants (TAs) for additional support outside class hours, and the experimental group, which used CodeTutor as their primary educational tool beyond class hours, alongside access to standard learning materials and human TAs. Using LLM-based tools other than CodeTutor in this course was prohibited.

To divide participants into a control group and an experimental group, we initially sorted the entire sample based on their previous engagement with LLM-powered tools, resulting in two groups: those who have used any LLM-powered tools before (*Used Before*)

and those who have not (*Never Used*). Within the *Used Before* category, we split the participants into two subsets, *Used Before Subset A* and *Used Before Subset B*, based on the overall pre-test result distribution to ensure both subsets are representative of the wider group. The same process was applied to the *Never Used* group, generating two additional subsets: *Never Used Subset A* and *Never Used Subset B*. The experimental group is then formed by combining *Used Before Subset A* with *Never Used Subset A*, while the control group consists of the combination of *Used Before Subset B* and *Never Used Subset B*. This method ensures the experimental and control groups were balanced regarding prior experience with Chatbots and their pre-test performance (see Figure 2).

Following their group assignments, students in the experimental group were sent detailed instructions via email on how to access and use CodeTutor. In the field study, participants were not mandated to adhere to a specific frequency of engagement with CodeTutor; instead, they were encouraged to utilize the tool at their own pace. This approach allowed for a naturalistic observation of how students integrate LLM-powered educational resources into their learning processes, without imposing additional constraints that could influence their study habits or the study’s outcomes.

3.3.3 Student Evaluation. At the end of the semester, students’ final grades were used as a primary measure to assess their learning outcomes and the impact of CodeTutor interventions. While acknowledging that final grades are influenced by various factors,

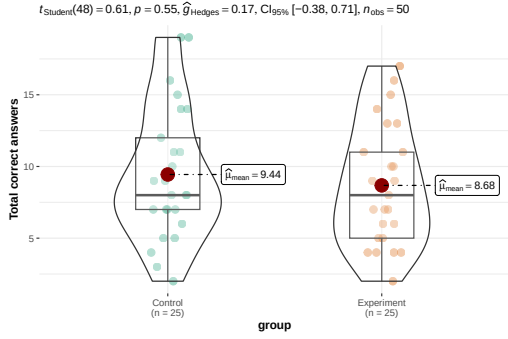


Figure 2: Parametric pairwise comparison (ANOVA) reveals no significant difference in correct answer count of pre-test in the control and experimental groups.

they offer a standardized measure of overall academic success, enabling an assessment of CodeTutor’s role in improving student learning outcomes.

Final grades were determined by a weighted average that includes several components for each student: labs (practical mini-projects), assignments (individual coding tasks, such as array summation), mid-terms, and a final exam (comprising questions similar to those in the pre-test). Note that a student’s final grade can surpass 100 if bonus points are awarded throughout the semester. Access to CodeTutor is restricted during mid-terms and final exams, categorizing the assessment components into two groups: *CodeTutor-Allowed* (labs and assignments) and *CodeTutor-Not-Allowed* (mid-terms and final exams). This categorization facilitates an analysis of CodeTutor’s impact on student performance by examining potential dependencies on the tool and the improvement of learning outcomes in its absence.

3.4 Data Analysis

3.4.1 Quantitative Data Analysis. We examined the students’ scores, interaction behaviors, and attitudes of using CodeTutor through multiple statistical analyses.

First, we calculated descriptive statistics for all variables, including frequency with percentage for categorical variables and means and standard deviations for continuous variables. To examine the variation in students’ scores before and after the intervention (i.e., the use of CodeTutor), we conducted *paired-t tests* for both the experimental and control groups. *Multiple regression analyses with family-wise p-value adjustment* were used to examine the effects of CodeTutor on score improvement, taking into account students’ past experiences using LLM-powered tools and demographic variables, such as major, gender, and race. We then investigated the impact of CodeTutor accessibility on academic performance with *ANOVA method*. Moreover, we conducted a *chi-squared test* to explore the relationship between the quality of students’ content and prompts and CodeTutor performance. To understand students’ attitudes towards CodeTutor, we calculated *Spearman’s correlation matrix for continuous variables*, given the characteristics of our data, which are non-normal and exhibit unequal variance. Furthermore, to examine differences between questions, we used the

Kruskal-Wallis Rank Sum Test (using R package stats [41]) and then performed post-hoc tests using *Dunnnett’s test* (using the R package FSA [30]) in cases where significant differences were found. To investigate the importance of time on students’ attitudes towards CodeTutor, we introduced a *linear mixed effects (LME) model* (using the R package lme4 [4]). We considered statistical significance at a significance level of $p < 0.05$ for most cases, except in multiple regression analyses where we used $p < 0.1$ and showed effect sizes were significant enough to indicate the relationship of variables.

3.4.2 Qualitative Data Analysis. We also analyzed the conversational history between users. Specifically, we used the *General Inductive Approach* [50] to guide our thematic analysis of the conversational data. The first author conducted a close reading of the data to gain a preliminary understanding of the conversational data and then labeled the text segments to formulate categories, which served as the basis for constructing low-level codes to capture specific elements of the user-CodeTutor interactions. Similar low-level codes were then clustered together to achieve high-level themes. During the analysis, the research team engaged in ongoing discussions to refine and clarify emerging themes.

4 RESULTS

In this section, we examined the impact of CodeTutor on student academic performance (subsection 4.1 to answer RQ1), analyzed students’ attitudes towards learning with CodeTutor (subsection 4.2 to answer RQ2), and characterized their engagement patterns in entry-level programming courses (subsection 4.3 to answer RQ3).

4.1 RQ1: Learning Outcomes with CodeTutor

4.1.1 Comparative Analysis of Score Improvements. Overall, students in the experimental group exhibited a greater average improvement in scores, as illustrated by comparing their pre-test and final scores to those in the control group. Specifically, the average increase for the experimental group was 12.50, whereas the control group showed an average decrease of 3.17 when comparing final scores to pre-test scores.

We conducted paired t-tests for both the experimental and control groups to determine if the observed improvements were statistically significant, starting with the premise that there were no differences in pre-test scores between these two groups. Our null hypothesis assumed that the true mean difference between pre-test and final scores was zero. For the control group, the null hypothesis could not be rejected, suggesting that the differences between pre-test and final scores were not statistically significant ($t = -0.879$, $p = 0.394$). Conversely, participants in the experimental group demonstrated significant improvement from the pre-test to final scores, indicating a statistically significant enhancement in their scores ($t = -2.847$, $p = 0.009$).

Furthermore, when examining the improvement in *CodeTutor-Not-Allowed* components, the experimental group exhibited an average increase of 7.33, whereas the control group showed no significant change. By conducting a paired t-test comparing the pre-test and final exam scores (during which the use of CodeTutor was not permitted), it was observed that students in the experimental group demonstrated a statistically significant improvement ($t = -2.405$, $p = 0.026$). This result suggests that students who have

Table 2: Multiple regression models explaining respondents' final scores in experimental group. (Significance level: $^{\dagger} p < 0.1$, $^* p < 0.05$, $^{} p < 0.01$, $^{***} p < 0.001$).**

	Estimate	Std. Error	t value	Pr(> t)
Const	93.683	3.877	24.166	0.000 ***
Prior Experiences with LLM tools				
(Reference: Used before)				
Never used	18.877	5.054	3.735	0.032 *
Major				
(Reference: Computer science)				
Data Science	14.532	5.662	2.567	0.073 †
Mathematics	17.692	5.852	3.023	0.057 †
Biology	16.257	5.662	2.871	0.057 †
Economics	1.362	4.799	0.284	0.784
Others	-13.004	6.022	-2.160	0.115
Gender				
(Reference: Female)				
Male	5.917	3.845	1.539	0.223
Race				
(Reference: White)				
Asian	-7.831	3.933	-1.991	0.128
African American or Black	8.099	7.107	1.140	0.322
Others	6.102	5.416	1.127	0.322

used CodeTutor exhibit more substantial improvement even when CodeTutor is unavailable.

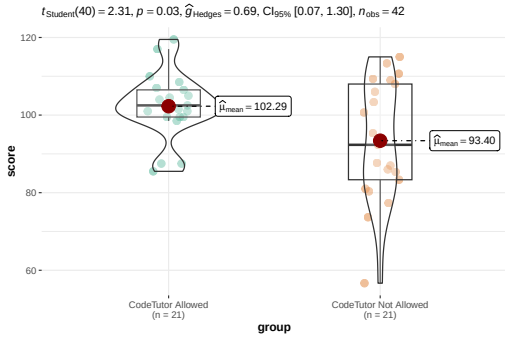


Figure 3: Parametric pairwise comparison (ANOVA) reveals a significantly higher mean score in the “CodeTutor-Allowed” group compared to the “CodeTutor-Not-Allowed” group.

4.1.2 Effect of CodeTutor Accessibility on Academic Performance. By constructing the *CodeTutor-Allowed* and *CodeTutor-Not-Allowed*, we determine the correlation between CodeTutor’s accessibility and student academic performance. Using the ANOVA technique on the data from the experimental group, Figure 3 reveals that the mean score for the *CodeTutor-Allowed* category stands at 102.29, in contrast to the *CodeTutor-Not-Allowed* components, which has a mean score of 93.40. The statistical analysis results show a significant difference between the two groups ($t = 2.31$, $p = 0.03$), suggesting that the allowance of CodeTutor correlates with higher student scores.

4.1.3 Correlation Between Student Demographics and Final Scores in the Experimental Group. Subsequently, we evaluated demographic

factors to determine whether specific student groups, particularly those with prior tech experience, experienced greater benefits from CodeTutor. Table 2 shows the results of multiple regression models, examining how students’ final scores in the experimental group are associated with their LLM history, major, gender, and race. Students who have never used any LLM-powered tools performed a significant increase ($\beta = 18.877$, $p = 0.032$) in final score than the students who used it before.

Moreover, differences in final scores among various majors within the experimental group were statistically significant, indicating that majors play a substantial role in final scores in the experimental group. Students majoring in data science ($\beta = 14.532$, $p = 0.073$), mathematics ($\beta = 17.692$, $p = 0.057$), and biology ($\beta = 16.257$, $p = 0.057$) exhibited a significant positive correlation with final scores compared to those majoring in computer science, suggesting that these majors achieved higher final scores. In terms of gender, no significant effects were observed, indicating no difference between genders in final scores. Additionally, no significant differences were noted across the races in final scores.

Summary of results of RQ1: Collectively, our findings suggest that students in the experimental group achieved significant score improvements with CodeTutor. Particularly, those who were new to CodeTutor achieved even greater improvements, while students majoring in data science, mathematics, and biology surpassed their computer science counterparts. Moreover, students exhibited higher scores when permitted to use CodeTutor.

4.2 RQ2: Students’ Attitudes towards CodeTutor

4.2.1 Descriptive Analysis. In terms of students’ attitudes towards CodeTutor (see Figure 1 for the specific questions), we found that a small portion of students (8%) strongly disagreed or disagreed that *CodeTutor accurately understood what students intended to ask*, while most (67%) agreed or strongly agreed. In addition, 35% strongly disagreed or disagreed that *CodeTutor helped them think critically*, while 19% agreed or strongly agreed. Furthermore, 13% students disagreed that *CodeTutor improved their understanding of programming syntax*, with a larger proportion of individuals agreeing (33%) or strongly agreeing (25%). Nearly half of the students (42%) agreed or strongly agreed that *CodeTutor helped students build their own understandings*, while very few (17%) strongly disagreed or disagreed. Finally, regarding *the potential of CodeTutor to substitute for a human teaching assistant*¹, 20% of the students strongly disagreed or disagreed with this notion, while 42% of them agreed or strongly agreed. Figure 4 shows the distribution of students’ responses across these five questions.

4.2.2 Exploring Relationships in Student Attitudes Toward CodeTutor. Figure 5 reveals key relationships among students’ attitudes on CodeTutor. The moderate positive correlation between Comprehension and Syntax Mastery suggests that proficiency in one is associated with higher performance in the other. Critical Thinking is slightly positive with Comprehension and Independent Learning but slightly negative with TA Replacement. Furthermore, Syntax Mastery strongly correlates with Independent Learning, indicating

¹In our analysis, response values to the TA Replacement question were reversed, so a higher score indicates a stronger preference for our tool over human teaching assistants. This reversal is consistently applied across all subsequent analyses.

Table 3: Linear Mixed-Effects Model of Student Attitudes over time. (Significance level: $^\dagger p < 0.1$, $* p < 0.05$, $ p < 0.01$, $*** p < 0.001$). Over time, students exhibit a significant decline in their agreement with CodeTutor’s comprehension and replacement of human teaching assistants.**

	Comprehension β (Std. Error)	Critical Thinking β (Std. Error)	Syntax Mastery β (Std. Error)	Independent Learning β (Std. Error)	TA Replacement β (Std. Error)
Const	4.700(0.297)***	2.690(0.247)***	3.760(0.262)***	3.044(0.218)***	3.964(0.330)***
Time	-0.114(0.039)**	0.040(0.037)	-0.018(0.041)	0.054(0.036)	-0.099(0.051) †

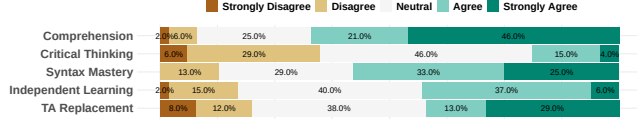


Figure 4: Participants’ attitudes toward CodeTutor, in terms of comprehension, critical thinking, syntax mastery, independent learning, and TA replacement (see Figure 1 for detailed questions).

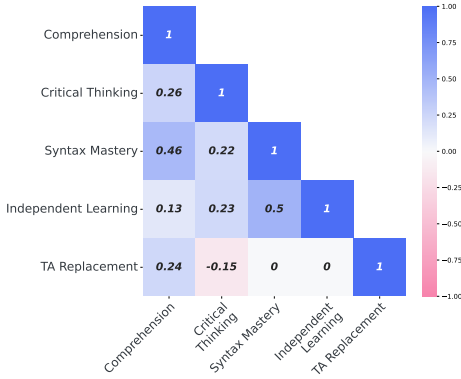


Figure 5: A correlation matrix heatmap visualizing the relationship between different metrics. The blue color indicates positive correlations, while pink represents negative correlations. Correlation coefficients are displayed inside each cell.

a close relationship between mastering programming syntax and self-directed learning outcomes. In addition, TA Replacement has minimal to no significant correlations with other variables, suggesting its effects vary independently of these educational aspects.

To further explore the relationship of different students’ attitudes among questions, we present the results of multiple comparisons across the five questions. Specifically, our results show that respondents’ attitudes ($\chi^2 = 32.99$, $p < 0.05$) significantly differ across questions. Our post-hoc tests (see Figure 6) further reveal that students were significantly less in agreement about CodeTutor’s assistance in fostering critical thinking compared to its ability to understand, help in learning syntax and serving as a replacement for a teaching assistant. Moreover, our findings suggest that respondents were significantly more in agreement with CodeTutor’s effectiveness in comprehension than in its ability to improve students’ understanding of programming syntax.

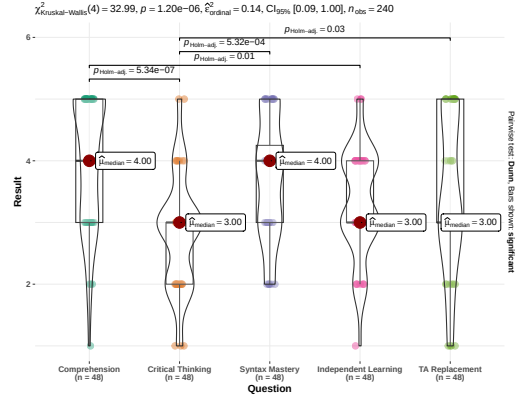


Figure 6: Non-parametric pairwise comparison test (Dunn’s test): Differences in agreement levels across different questions. We can see that students predominantly favored CodeTutor for its comprehension and syntax support rather than its ability to foster critical thinking. Additionally, there was a stronger consensus on CodeTutor’s proficiency in understanding queries compared to its effectiveness in enhancing programming syntax.

We then conducted a linear mixed effects (LME) model to explore time’s influence on students’ attitudes toward CodeTutor:

$$QuestionIndicator_{it} = \beta_0 + b_{0i} + (\beta_1 + b_{1i})t + \epsilon_{it}$$

where β_0 and β_1 are unknown fixed effect parameters; b_{0i} and b_{1i} are the unknown student-specific random intercept and slope, respectively, which are assumed to have a bivariate normal distribution with mean zero and covariance matrix D ; $QuestionIndicator$ is the student response at time t ; and ϵ_{it} is the residual error for student i at time t , with a normal distribution $N(0, \sigma^2)$, which is assumed to be independent of the random effects. From Table 3, we can see that students’ attitudes toward CodeTutor show a significant decrease in Comprehension ($\beta = -0.114$, $p < 0.01$), which indicates that students disagree with CodeTutor’s understanding accuracy over time. Moreover, there is a weakly significant decrease in TA Replacement ($\beta = -0.099$, $p < 0.1$) with increasing time. This shows a slight tendency for them to consider more human TA help over time. Also, students perform no significant difference over time in Critical Thinking, Syntax Mastery, and Independent Learning.

Summary of results of RQ2: In summary, students recognize CodeTutor’s ability to understand their queries and assist with programming syntax yet question its capacity to promote critical thinking skills. Additionally, students’ confidence in CodeTutor’s

comprehension abilities decreases over time, with a growing preference for support from human teaching assistants.

4.3 RQ3: Students' Engagement with CodeTutor

In total, we documented 82 conversation sessions² with CodeTutor, encompassing a total of 2,567 messages. In these sessions, 415 unique topics were discussed, averaging 5.06 topics per session and 6.19 messages per topic.

4.3.1 Message Classification & Interaction Patterns. In total, we collected 2567 conversational messages exchanged between users and the CodeTutor. Of these, 1288 messages originated from the users, and CodeTutor responded with 1279 messages.

Table 4 presents categorizations of messages between users and CodeTutor. Each category has a description and an example to illustrate the message type. Categories of messages from both users and CodeTutor include *Programming Task inquiries*, addressing specific Python programming challenges; *Grammar and Syntax questions*, focusing on Python's basic grammar or syntax without necessitating runnable programs; *General Questions*, which are not directly related to Python; and *Greetings*, initiating or finishing interaction.

From the users' side, additional categories highlight their engagement with CodeTutor: *Modification Requests* for alterations to previous answers; *Help Ineffective* indicating issues or errors in CodeTutor's provided solutions; *Further Information* to elaborate on prior queries; and *Debug Requests* for assistance in resolving bugs or errors in code snippets.

CodeTutor's responses are classified into *Corrections*, which address and amend errors in previous responses and *Explanations*, providing further details on provided solutions or clarify why certain requests cannot be fulfilled.

4.3.2 Analysis of Prompt Quality & Correlation with Response Effectiveness. To further examine user interaction patterns with CodeTutor and their implications for its educational value, we analyzed the relationship between prompt quality and response accuracy. This analysis stems from the premise that detailed and precise prompts are likely to improve the AI's understanding of user requirements, thereby potentially raising the standard of its responses.

To do so, we evaluated a corpus of 1,190 prompts, after removing all greeting messages, to assess their quality. Our analysis showed that 37% were deemed good quality. The remaining 63% were identified as poor quality. We defined "good quality" prompts as providing sufficient detail for CodeTutor to generate an accurate response. In contrast, "poor quality" prompts were those that did not meet this criterion. We categorized the deficiencies in poor quality prompts into four types: incomplete information ($n = 189$, 25%), which lacked specific details necessary for CodeTutor to understand the context; lack of clear goals ($n = 172$, 23%), where the desired outcome was not explicitly stated; over-reliance on CodeTutor ($n = 362$, 48%), where the assignment questions are directly copied and pasted into CodeTutor; and poor structural organization ($n = 25$, 3%), which exhibited unclear or confusing request structures. Prompts were

further labeled as "working" if they elicited an appropriate response from CodeTutor, and "not working" if they failed to do so.

Using a Chi-square test, we investigated whether the prompt quality and the effectiveness of CodeTutor's responses were independent. Our results showed a significant correlation ($\chi^2 = 144.84$, $p < 0.001$). In other words, clearer and more detailed prompts are associated with responses that are more likely to be effective.

Summary of results of RQ3: We characterized the messages exchanged between users and CodeTutor. We categorize these interactions between users and CodeTutor into inquiries (e.g., programming tasks, syntax questions) and feedback alongside CodeTutor's responses (corrections and explanations), illustrating a dynamic exchange aimed at facilitating learning. We also found that the clarity and completeness of prompts are significantly correlated with the quality of responses from CodeTutor.

5 DISCUSSION

Our semester-long field study provided insights into how students in introductory computer science courses utilized CodeTutor and its effects on educational outcomes. In short, our results show that 1) students who used CodeTutor had shown significant improvements in scores; 2) while CodeTutor was valued for its assistance in comprehension and syntax, students expressed concerns about its capacity to enhance critical thinking skills; 3) skepticism regarding CodeTutor as an alternative to human teaching assistants grew over time; 4) CodeTutor was primarily used for various coding tasks, including syntax comprehension, debugging, and clarifying fundamental concepts; 5) the effectiveness of CodeTutor responses was notably higher when prompts were clearer and more detailed. Building on these findings, we discuss the implications for future enhancements and research directions in the rest of the section.

5.1 Towards Enhancing Generative AI Literacy

Our research indicates a positive correlation between using Generative AI tools and improved student learning outcomes. However, 63% of student-generated prompts were deemed unsatisfactory, indicating a lack of essential skills to fully exploit Generative AI tools. This finding also suggests the need to promote *Generative AI literacy* among students. Here, we define *Generative AI literacy* as the ability to be aware of, recognize, understand, and critically engage with Generative AI technologies, which is important for understanding the potential of Generative AI and for navigating its multifaceted ethical and societal implications. Our findings suggest that while students can leverage CodeTutor for practical coding assistance and syntax understanding, there is a gap in using these tools to enhance critical thinking skills. We suggest educational programs integrate Generative AI literacy as a core component of their curriculum, teaching students how to use these tools for immediate problem-solving and engaging with them to promote deeper analytical and critical thinking. This could include workshops on effective query formulation, sessions on interpreting AI responses, and exercises designed to challenge students to critically evaluate the information and solutions offered by AI tools.

We also propose approaches to integrate HCI tools and principles into LLM-enabled platforms, such as *prompt construction templates* providing users with templates or structured forms for crafting

²In our analysis, a conversation session is a continuous exchange of messages between users and CodeTutor within a specific period, characterized by a coherent topic or purpose.

Table 4: Categorizations of messages, from users’ side ⑧ and from CodeTutor’s side ⑨. [Code Snippet] represents a Python code segment. The *Percentage* column represents the ratio of occurrences of each category to the total number of messages. Note that some categories may only apply to messages sent by either users or CodeTutor, and messages may carry multiple categories.

Category Name	Description	Example	Percentage
⑧⑨ Programming Task	Any questions or answers related to Python programming.	<i>“Write a function that prints the nth(argument) prime number.”</i>	86.52%
⑧⑨ Grammar & Syntax	When a message is related to basic Python grammar or syntax problems, a runnable program is most likely unnecessary.	<i>“What does {} do in Python?”</i>	14.26%
⑧⑨ General Question	When a message is not directly related to Python.	<i>“What is ASCII?”</i>	4.29%
⑧⑨ Greetings	When a message is greeting.	<i>“Hello! How can I assist you today?”</i>	0.62%
⑧ Help Ineffective	When a user message says the previous answer generated by CodeTutor is wrong or provides error information.	<i>“This code still fails.”</i>	12.86%
⑧ Debug Request	When a user message asks CodeTutor to fix bugs or explain what was wrong in code snippets provided or in previous messages.	<i>“Debug this code. [Code Snippet]”</i>	8.22%
⑧ Modification Request	When a user requires CodeTutor to change something on its previous answer.	<i>“Remove comments.”</i>	4.48%
⑧ Further Information	When a user message provides more context on their previous input.	<i>“All the input strings will be the same length.”</i>	3.97%
⑨ Explanation	When CodeTutor explains something in previous messages or why it cannot complete the current task from users.	<i>“I’m sorry, but I need more information to provide the answers for questions 4 and 6.”</i>	28.94%
⑨ Correction	When CodeTutor corrects content in its previous answer.	<i>“Apologies for the syntax error. Here is the corrected version: [Code Snippet]”</i>	13.95%

queries. They can guide users in formulating more effective and precise questions. Templates could include placeholders for essential details and context, providing the necessary information for the AI to generate accurate responses to users. Furthermore, integrating *Critical Thinking Prompts* might be particularly effective in stimulating in-depth analytical thinking. For example, the interface could pose follow-up questions encouraging users to assess AI answers’ adequacy critically. Questions such as, “Does this response fully address your query?” or “What additional information might you need?” may prompt users to engage in a more thorough evaluation of the information provided, fostering a habit of critical reflection and assessment. Another possible approach is *Facilitating Collaborative Query Building*, which leverages the power of collective intelligence. By designing interfaces that support real-time collaboration among users, individuals can work together to construct and refine queries. We can also use LLMs to evaluate and refine user questions instantly as they perform well in prompting [60].

5.2 Turning to the Temporal Dynamics of LLM-Powered Tutoring Tools

The temporality aspect of using CodeTutor in computer science education presents a nuanced perspective on their integration and effectiveness over time. Our analysis reveals a complex relationship between the duration of CodeTutor use and students’ attitudes towards it. Specifically, our results show that although students initially find CodeTutor a reliable tool for understanding their queries, their confidence in its accuracy diminishes with prolonged use. Additionally, our model uncovers a weakly significant decrease in students’ preference for CodeTutor as a TA replacement over time. This trend implies a growing inclination among students to seek human TA support as they progress in their courses, possibly due to

the nuanced understanding and personalized feedback that human TAs can offer, which might not be fully replicated by LLMs. However, our study found no significant temporal change in students’ attitudes toward CodeTutor’s impact on critical thinking, syntax mastery, and independent learning. This stability suggests that while students may question CodeTutor’s comprehension abilities and its adequacy as a TA replacement over time, they still recognize its utility in facilitating certain aspects of the learning process, such as mastering syntax and promoting independent study habits.

Collectively, our findings highlight the importance of investigating the temporal dynamics of student attitudes towards and their use of LLM-powered tools for learning and shed light on the need for a balanced approach to integrating LLMs into CS education. While these tools offer great support in specific areas, their limitations become more apparent with extended use. In other words, it is important to complement LLMs with human instruction to address learning objectives, such as critical thinking and problem-solving, which are crucial for computer science education. Furthermore, we argue that educators and developers should work collaboratively to enhance the capabilities of LLM-powered tutoring systems, ensuring they remain effective and relevant over time.

5.3 Alignments of LLMs for Education

Our observations regarding students’ utilization of CodeTutor provide insights into their learning approaches and completion of assignments. The exams that prohibit using CodeTutor reflect students’ understanding of programming, as they must rely solely on their internal knowledge. Conversely, assignments and lab tasks that permit using CodeTutor result in higher scores, indicating that students may prioritize completion over deep comprehension [17].

While students employ CodeTutor to fulfill homework requirements, they may not perceive it as a tool for a comprehensive understanding of course materials.

Our results show that nearly half of the low-quality prompts classified as *over-reliance* were copied and pasted original assignment questions into CodeTutor. This suggests that students primarily used CodeTutor as a quick-fix solution, neglecting the opportunity to engage with the underlying question logic and determine appropriate solutions to the question. As the complexity of assignments increased, students' perceptions of CodeTutor's ability to understand their queries turned more negative. However, students acknowledge its proficiency in syntax mastery, which reveals a gap between their expectations and the tool's capabilities. Complex questions require students to integrate and apply the knowledge acquired in class [51], challenging the notion that CodeTutor can easily break down questions into manageable components. Additionally, CodeTutor's limitations, such as its training on a predetermined database and inability to handle custom or complex queries, suggest that it is important to simplify questions and structure prompts effectively for optimal results.

Furthermore, we argue that students' previous experiences with chatbots, if unrelated to structured learning, such as a simple one-line request (e.g., "help me write a summary"), may not adequately prepare them for using CodeTutor effectively in a programming context, as evidenced by our findings that nearly 70% of student submissions in our corpus were of poor quality. Students with limited experience interacting with chatbots might be hesitant to trust tools like CodeTutor fully, potentially affecting their use and reliance on its outputs. This lack of familiarity could lead them to prefer traditional learning approaches, fostering deeper analytical thinking and minimizing dependency on automated assistance.

Design Implications. Our findings shed light on the future implementation and enhancement of CodeTutor in programming courses. The inherent limitations of CodeTutor, which is trained on a general dataset, may necessitate the creation of custom datasets tailored to specific class contexts. Through instructors' reflections on the quality of students' assignments, it becomes evident that while CodeTutor produces impressive results due to its training on datasets crafted by professional programmers aimed at efficiency, the emphasis in entry-level classes should prioritize human-readable code over complex solutions. One potential solution is to leverage GPT models with the Assistant API [31]. This API enables the development of AI assistants with features, such as the Code Interpreter [33], which can execute Python code in a sandboxed environment, and Knowledge Retrieval [34], allowing users to upload documents to enhance the assistant's knowledge base. These features align more closely with the requirements of a virtual TA in entry-level programming courses. The Code Interpreter can enhance the quality of responses containing code blocks, while Knowledge Retrieval empowers instructors to provide course-specific information. Meanwhile, providing systematic instructions to students can enhance their understanding of how to use the tool effectively while improving its accessibility through additional instructional features. Additionally, it is crucial to emphasize the boundaries of using LLM-powered tools, clarifying what is permissible and the consequences of inappropriate usage. We should adapt LLMs to support educational objectives by offering appropriate

guidance and instructional content tailored for students, ensuring information accuracy, and promoting critical thinking over the mere provision of direct answers.

6 LIMITATIONS AND FUTURE WORK

Our study, while providing valuable insights into the use of LLM-powered tools in educational settings, has several limitations that suggest avenues for further research. First, The current study was conducted on a relatively small scale, limiting the generalizability of our findings. Therefore, our future work will conduct larger-scale tests involving more diverse student populations and settings. Second, regarding the applicability to different levels of coding courses, our work has focused on beginning levels of CS courses. Our findings may not directly translate to intermediate or advanced programming courses. Furthermore, we relied on GPT-3.5 in this study, which may not always provide accurate or contextually appropriate responses, potentially affecting the quality of tutoring provided. Lastly, controlling the experimental environment in a semester-long study, particularly the control group, was challenging, indicating the need for more experimental designs in future studies to better understand the factors affecting student learning.

7 CONCLUSION

In this work, we conducted a semester-long between-subjects study with 50 students to examine the ways in which students use an LLM-powered virtual teaching assistant (i.e., CodeTutor) in their introductory-level programming learning. The experimental group using CodeTutor showed significant improvements in final scores over the control group, with first-time users of LLM-powered tools experiencing the most substantial gains. While positive feedback was received on CodeTutor's ability to understand queries and aid in syntax learning, concerns were raised about its effectiveness in cultivating critical thinking skills. Over time, we observed a shift towards preferring human assistant support over CodeTutor, despite its utility in completing programming tasks, understanding syntax, and debugging. Our study also shows the importance of prompt quality in leveraging CodeTutor's effectiveness, indicating that detailed and clear prompts yield more accurate responses. Our findings point to the critical need for embedding *Generative AI literacy* into educational curricula and to promote critical thinking abilities among students. Looking ahead, our research suggests integrating LLM-powered tools in computer science education requires more tools, resources, and regulations to help students develop Generative AI literacy and customize teaching strategies to bridge the gap between tool capabilities and educational goals. By adjusting expectations and guiding students on effective tool use, educators may harness the full potential of Generative AI to complement traditional teaching methods.

ACKNOWLEDGMENTS

This project is funded by the Studio for Teaching & Learning Innovation Learn, Discover, Innovate Grant, the Faculty Research Grant from William & Mary, and the Microsoft Accelerate Foundation Models Research Award. We thank our participants in this study and our anonymous reviewers for their feedback.

REFERENCES

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). <https://doi.org/10.48550/arXiv.2303.08774>
- [2] Toufique Ahmed, Noah Rose Ledesma, and Premkumar Devanbu. 2022. SYN-SHINE: improved fixing of syntax errors. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2169–2181. <https://doi.org/10.1109/TSE.2022.3212635>
- [3] John R Anderson, C Franklin Boyle, and Brian J Reiser. 1985. Intelligent tutoring systems. *Science* 228, 4698 (1985), 456–462. <https://doi.org/10.1126/science.228.4698.456>
- [4] Douglas Bates, Martin Mächler, Ben Bolker, and Steve Walker. 2015. Fitting Linear Mixed-Effects Models Using lme4. *Journal of Statistical Software* 67, 1 (2015), 1–48. <https://doi.org/10.18637/jss.v067.i01>
- [5] Peter Brusilovsky et al. 1998. Adaptive educational systems on the world-wide-web: A review of available technologies. In *Proceedings of Workshop "WWW-Based Tutoring" at 4th International Conference on Intelligent Tutoring Systems (ITS'98)*, San Antonio, TX.
- [6] Peter Brusilovsky, Elmar Schwarz, and Gerhard Weber. 1996. ELM-ART: An intelligent tutoring system on World Wide Web. In *Intelligent Tutoring Systems: Third International Conference, ITS'96 Montréal, Canada, June 12–14, 1996 Proceedings* 3. Springer, 261–269. https://doi.org/10.1007/3-540-61327-7_123
- [7] Cory J Butz, Shan Hua, and R Brien Maguire. 2006. A web-based bayesian intelligent tutoring system for computer programming. *Web Intelligence and Agent Systems: An International Journal* 4, 1 (2006), 77–97.
- [8] Jan Clusmann, Fiona R Kolbinger, Hannah Sophie Muti, Zunamys I Carrero, Jan-Niklas Eckardt, Narmin Ghaffari Laleh, Chiara Maria Lavinia Löffler, Sophie-Caroline Schwarzkopf, Michaela Unger, Gregory P Veldhuizen, et al. 2023. The future landscape of large language models in medicine. *Communications Medicine* 3, 1 (2023), 141. <https://doi.org/10.1038/s43856-023-00370-1>
- [9] Albert T Corbett, Kenneth R Koedinger, and John R Anderson. 1997. Intelligent tutoring systems. In *Handbook of human-computer interaction*. Elsevier, 849–874. <https://doi.org/10.1016/B978-044481862-1.50103-5>
- [10] Dorothy Demszyk and Jing Liu. 2023. M-Powering Teachers: Natural Language Processing Powered Feedback Improves 1:1 Instruction and Student Outcomes. (2023). <https://doi.org/10.1145/3573051.3593379>
- [11] Paul Denny, Sami Sarsa, Arto Hellas, and Juho Leinonen. 2022. Robosourcing Educational Resources—Leveraging Large Language Models for Learnersourcing. *arXiv preprint arXiv:2211.04715* (2022). <https://doi.org/10.1145/3501385.3543957>
- [12] Felix Dobszlaw and Peter Bergh. 2023. Experiences with Remote Examination Formats in Light of GPT-4. *arXiv preprint arXiv:2305.02198* (2023). <https://doi.org/10.48550/arXiv.2305.02198>
- [13] Gilan M El Saadawi, Eugene Tseytlin, Elizabeth Legowski, Drazen Jukic, Melissa Castine, Jeffrey Fine, Robert Gormley, and Rebecca S Crowley. 2008. A natural language intelligent tutoring system for training pathologists: Implementation and evaluation. *Advances in health sciences education* 13 (2008), 709–722. <https://doi.org/10.1007/s10459-007-9081-3>
- [14] Mark Elsom-Cook. 1984. *Design considerations of an intelligent tutoring system for programming languages*. Ph. D. Dissertation. University of Warwick.
- [15] GitHub, Inc. 2024. GitHub Copilot. <https://github.com/features/copilot>. Accessed: 2024-02-11.
- [16] Arthur C Graesser, Xiangen Hu, and Robert Sottilare. 2018. Intelligent tutoring systems. In *International handbook of the learning sciences*. Routledge, 246–255.
- [17] Morgan Gustafson. 2022. The Effect of Homework Completion on Students' Academic Performance. Dissertations, Theses, and Projects. https://red.mnstate.edu/thesis/662_662
- [18] Yann Hicce, Anmol Agarwal, Qianou Ma, and Paul Denny. 2023. ChaTA: Towards an Intelligent Question-Answer Teaching Assistant using Open-Source LLMs. *arXiv preprint arXiv:2311.02775* (2023). <https://doi.org/10.48550/arXiv.2311.02775>
- [19] Danial Hooshyar, Rodina Binti Ahmad, Moslem Yousefi, Farrah Dina Yusop, and S-J Horng. 2015. A flowchart-based intelligent tutoring system for improving problem-solving skills of novice programmers. *Journal of computer assisted learning* 31, 4 (2015), 345–361. <https://doi.org/10.1111/jcal.12099>
- [20] Sajed Jalil, Suzzana Rafi, Thomas D LaToza, Kevin Moran, and Wing Lam. 2023. Chatgpt and software testing education: Promises & perils. In *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 4130–4137. <https://doi.org/10.1109/ICSTW58534.2023.00078>
- [21] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günemann, Eyke Hüllermeier, et al. 2023. ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and individual differences* 103 (2023), 102274. <https://doi.org/10.1016/j.lindif.2023.102274>
- [22] James A Kulik and JD Fletcher. 2016. Effectiveness of intelligent tutoring systems: a meta-analytic review. *Review of educational research* 86, 1 (2016), 42–78. <https://doi.org/10.3102/0034654315581420>
- [23] Harsh Kumar, Ilya Musabirov, Mohi Reza, Jiakai Shi, Anastasia Kuzminykh, Joseph Jay Williams, and Michael Liut. 2023. Impact of Guidance and Interaction Strategies for LLM Use on Learner Performance and Perception. *arXiv preprint arXiv:2310.13712* (2023). <https://doi.org/10.48550/arXiv.2310.13712>
- [24] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing code explanations created by students and large language models. *arXiv preprint arXiv:2304.03938* (2023). <https://doi.org/10.48550/arXiv.2304.03938>
- [25] Juho Leinonen, Arto Hellas, Sami Sarsa, Brent Reeves, Paul Denny, James Prather, and Brett A Becker. 2023. Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 563–569. <https://doi.org/10.1145/3545945.3569770>
- [26] Mark Liffiton, Brad E Sheese, Jaromir Savelka, and Paul Denny. [n. d.]. Codehelp: Using large language models with guardrails for scalable support in programming classes. ([n. d.]), 1–11. <https://doi.org/10.1145/3631802.3631830>
- [27] Atharva Mehta, Nipun Gupta, Dhruv Kumar, Pankaj Jalote, et al. 2023. Can ChatGPT Play the Role of a Teaching Assistant in an Introductory Programming Course? *arXiv preprint arXiv:2312.07343* (2023). <https://doi.org/10.48550/arXiv.2312.07343>
- [28] Jesse G Meyer, Ryan J Urbanowicz, Patrick CN Martin, Karen O'Connor, Ruowang Li, Pei-Chen Peng, Tiffani J Bright, Nicholas Tatonetti, Kyoung Jae Won, Graciela Gonzalez-Hernandez, et al. 2023. ChatGPT and large language models in academia: opportunities and challenges. *BioData Mining* 16, 1 (2023), 20. <https://doi.org/10.1186/s13040-023-00339-9>
- [29] Hyacinth S Nwana. 1990. Intelligent tutoring systems: an overview. *Artificial Intelligence Review* 4, 4 (1990), 251–277. <https://doi.org/10.1007/BF00168958>
- [30] Derek H. Ogle, Jason C. Doll, A. Powell Wheeler, and Alexis Dinno. 2023. FSA: Simple Fisheries Stock Assessment Methods. <https://CRAN.R-project.org/package=FSA> R package version 0.9.4.
- [31] OpenAI. 2024. Assistants Overview - OpenAI API. <https://platform.openai.com/docs/assistants/overview>. Accessed: 2024-02-11.
- [32] OpenAI. 2024. ChatGPT. <https://openai.com/chatgpt>. Accessed: 2024-02-11.
- [33] OpenAI. 2024. Code Interpreter. <https://platform.openai.com/docs/assistants/tools/code-interpreter>. Accessed: 2024-02-11.
- [34] OpenAI. 2024. Knowledge Retrieval. <https://platform.openai.com/docs/assistants/tools/knowledge-retrieval>. Accessed: 2024-02-11.
- [35] Maciej Pankiewicz and Ryan S Baker. 2023. Large Language Models (GPT) for automating feedback on programming assignments. *arXiv preprint arXiv:2307.00150* (2023). <https://doi.org/10.48550/arXiv.2307.00150>
- [36] Mike Perkins, Jasper Roe, Darius Postma, James McGaughan, and Don Hickerson. 2023. Detection of GPT-4 generated text in higher education: Combining academic judgement and software to identify generative AI tool misuse. *Journal of Academic Ethics* (2023), 1–25. <https://doi.org/10.1007/s10805-023-09492-6>
- [37] Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating High-Precision Feedback for Programming Syntax Errors using Large Language Models. *arXiv preprint arXiv:2302.04662* (2023). <https://doi.org/10.48550/arXiv.2302.04662>
- [38] Tung Phung, Victor-Alexandru Pădurean, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generative AI for Programming Education: Benchmarking ChatGPT, GPT-4, and Human Tutors. *International Journal of Management* 21, 2 (2023), 100790. <https://doi.org/10.48550/arXiv.2306.17156>
- [39] Russell A Poldrack, Thomas Lu, and Gasper Beguš. 2023. AI-assisted coding: Experiments with GPT-4. *arXiv preprint arXiv:2304.13187* (2023). <https://doi.org/10.48550/arXiv.2304.13187>
- [40] James Prather, Paul Denny, Juho Leinonen, Brett A Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, et al. 2023. The robots are here: Navigating the generative ai revolution in computing education. *arXiv preprint arXiv:2310.00658* (2023). <https://doi.org/10.1145/3623762.3633499>
- [41] R Core Team. 2022. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>
- [42] Steven Ritter, John R Anderson, Kenneth R Koedinger, and Albert Corbett. 2007. Cognitive Tutor: Applied research in mathematics education. *Psychonomic bulletin & review* 14 (2007), 249–255. <https://doi.org/10.3758/BF03194060>
- [43] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 1*. 27–43.
- [44] Jaromir Savelka, Arav Agarwal, Christopher Bogart, and Majd Sakr. 2023. Large language models (gpt) struggle to answer multiple-choice questions about code. *arXiv preprint arXiv:2303.08033* (2023). <https://doi.org/10.48550/arXiv.2303.08033>
- [45] Brad Sheese, Mark Liffiton, Jaromir Savelka, and Paul Denny. 2023. Patterns of Student Help-Seeking When Using a Large Language Model-Powered Programming Assistant. *arXiv preprint arXiv:2310.16984* (2023). <https://doi.org/10.1145/3636243.3636249>
- [46] Derek Sleeman and John Seely Brown. 1982. *Intelligent tutoring systems*. London: Academic Press.

- [47] Robert A Sottolare, Keith W Brawner, Benjamin S Goldberg, and Heather K Holden. 2012. The generalized intelligent framework for tutoring (GIFT). *Orlando, FL: US Army Research Laboratory–Human Research & Engineering Directorate (ARL-HRED)* (2012).
- [48] Lichao Sun, Yue Huang, Haoran Wang, Siyuan Wu, Qihui Zhang, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, Xiner Li, et al. 2024. Trustllm: Trustworthiness in large language models. *arXiv preprint arXiv:2401.05561* (2024). <https://doi.org/10.48550/arXiv.2401.05561>
- [49] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large language models in medicine. *Nature medicine* 29, 8 (2023), 1930–1940. <https://doi.org/10.1038/s41591-023-02448-8>
- [50] David R Thomas. 2006. A general inductive approach for analyzing qualitative evaluation data. *American journal of evaluation* 27, 2 (2006), 237–246. <https://doi.org/10.1177/1098214005283748>
- [51] Ulrich Trautwein and Olaf Köller. 2003. The relationship between homework and achievement—still much of a mystery. *Educational psychology review* 15 (2003), 115–145. <https://doi.org/10.1023/A:1023460414243>
- [52] Kurt VanLehn. 2011. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational psychologist* 46, 4 (2011), 197–221. <https://doi.org/10.1080/00461520.2011.611369>
- [53] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. 2022. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682* (2022). <https://doi.org/10.48550/arXiv.2206.07682>
- [54] Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Derek F Wong, and Lidia S Chao. 2023. A survey on llm-generated text detection: Necessity, methods, and future directions. *arXiv preprint arXiv:2310.14724* (2023). <https://doi.org/10.48550/arXiv.2310.14724>
- [55] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. 2023. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564* (2023). <https://doi.org/10.48550/arXiv.2303.17564>
- [56] JD Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–21. <https://doi.org/10.1145/3544548.3581388>
- [57] Yixuan Zhang, Yimeng Wang, Nutchanon Yongsatianchot, Joseph D Gaggiano, Nurul M Suhaimi, Anne Okrah, Miso Kim, Jacqueline Griffin, and Andrea G Parker. 2024. Profiling the Dynamics of Trust & Distrust in Social Media: A Survey Study. (2024). <https://doi.org/10.1145/3613904.3642927>
- [58] Jiawei Zhou, Yixuan Zhang, Qianni Luo, Andrea G Parker, and Munmun De Choudhury. 2023. Synthetic lies: Understanding ai-generated misinformation and evaluating algorithmic and human solutions. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–20. <https://doi.org/10.1145/3544548.3581318>
- [59] Kyrie Zhixuan Zhou, Zachary Kilhoffer, Madelyn Rose Sanfilippo, Ted Underwood, Ece Gumusel, Mengyi Wei, Abhinav Choudhry, and Jinjun Xiong. 2024. "The teachers are confused as well": A Multiple-Stakeholder Ethics Discussion on Large Language Models in Computing Education. *arXiv preprint arXiv:2401.12453* (2024). <https://doi.org/10.48550/arXiv.2401.12453>
- [60] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitit, Harris Chan, and Jimmy Ba. 2022. Large language models are human-level prompt engineers. *arXiv preprint arXiv:2211.01910* (2022). <https://doi.org/10.48550/arXiv.2211.01910>